DR. MPS MEMORIAL COLLEGE OF BUSINESS STUDIES

Sikandra, Agra

(Affiliated to Dr. Bhimrao Ambedkar University, Agra)

PROCRASTIFY

An Anti-Procrastination and Productivity Management System

Stop Procrastinating, Start Achieving.

A PROJECT REPORT

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF COMPUTER APPLICATION

(Session 2023 – 2026)

Under the Guidance of:

Mr. Vijay Bhadouria
Faculty Member,
Department of Computer Science

Submitted by:

Saksham
Roll No. _____

Vivek Shukla
Roll No. _____

DEPARTMENT OF COMPUTER SCIENCE

Dr. MPS Memorial College of Business Studies, Sikandra, Agra

2026

CERTIFICATE

Certified that the project report titled "Procrastify: An Anti-Procrastination and Productivity Management System" is the original work of Saksham, Roll No. _____, and Vivek Shukla, Roll No. _____, submitted to Dr. Bhimrao Ambedkar University, Agra, for the award of the degree of Bachelor of Computer Application. This is a record of work carried out by them under my supervision. This work has not been submitted for the award of any other degree.

_____

Mr. Vijay Bhadouria

Faculty Member, Department of Computer Science

Dr. MPS Memorial College of Business Studies, Sikandra, Agra

_____

Dean

Department of Computer Science

DECLARATION

We, Saksham and Vivek Shukla, hereby declare that the project entitled "Procrastify: An Anti-Procrastination and Productivity Management System" is our original work. To the best of our knowledge and belief, it does not contain any material previously written by another person, and no part of it has been submitted for the award of any other degree or diploma of any university or institute of higher learning, except where proper acknowledgement has been made in the text.

Date: _____

Place: Agra

Saksham
Vivek Shukla
B.C.A – 6th Semester

ACKNOWLEDGEMENT

We are very thankful to our project supervisor Mr. Vijay Bhadouria, Faculty Member, Department of Computer Science, for guiding us through the entire project. He gave us direction whenever we were confused and his suggestions helped us improve the project at different stages. Above all, he provided us with unflinching encouragement and support throughout the work.

We would also like to extend special thanks to Director Academics Dr. Vikrant Shastri, Director Administration Dr. A. K. Goyal, and Dean Computer Science Dr. Amendra Singh for their support and advice throughout the project.

A special thanks goes to our families. There were many late nights during this project when things were not working and we felt mentally tired. Their patience and encouragement helped us continue.

We also want to thank our classmates who gave honest feedback during presentations. That feedback helped us understand what needed improvement and what parts of the project were already strong.

Finally, we would like to thank everybody who was important to the successful completion of this project.

Saksham
Vivek Shukla

ABSTRACT

For our final year project, we built a web application called Procrastify. We chose this because we realised that procrastination is not actually a personality flaw, it is really an information problem. Students and working professionals do not fail to start working because they are lazy. They fail because they open a normal to-do app, see a huge flat list of tasks, get overwhelmed, and then end up scrolling on Instagram instead. Procrastify fixes this by giving users a clear, dynamically ranked view of exactly what they need to do first.

The feature we are most proud of is our custom Priority Scoring Logic. Most productivity apps sort tasks by due date alone, which is an incomplete picture. We created a two-variable formula that divides the days remaining until the deadline by the task complexity rating. The result is what we call the Priority Score, the lower the score, the higher the urgency. A task that is overdue gets a score of zero and is immediately flagged. The system also colour-codes everything into five levels: Overdue, High, Medium, Low, and Relaxed, and it recalculates live every time the dashboard is loaded.

Once a user knows what to do, the application provides a built-in Pomodoro focus timer linked directly to the selected task. The default session is 15 minutes, with options for 25, 45, and 60 minutes. After each session, users self-report whether the session was completed or interrupted. All this data feeds into an analytics dashboard that calculates a live Focus Score, which is essentially productive time divided by total tracked time, expressed as a percentage.

The analytics module also includes a gamified streak tracker that levels users up from Starting to On Fire to Legendary the more consistently they complete focus sessions. This makes the app feel rewarding rather than just restrictive.

For the technical stack, we built the backend using Python, Flask, and MySQL, and used Streamlit to create a 5-page frontend interface. The frontend and backend are completely decoupled and communicate through a REST API with JWT authentication and bcrypt password hashing for security. The web application is fully functional on a local server.

Basically, Procrastify does not just remind you what to do. It tells you what to do first and holds you accountable while you do it.

TABLE OF CONTENTS

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

List of Figures v

List of Tables vi

1. Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives 3

1.4 Scope of the Project 3

1.5 Methodology 4

1.6 Organisation of the Report 4

2. Literature Review 5

2.1 Previous Work 5

2.2 Related Studies 6

2.3 Summary 7

3. System Analysis / Project Planning 8

3.1 Requirements Gathering 8

3.2 Feasibility Study 9

3.3 Tools and Technologies Used 10

3.4 Risk Analysis 11

4. System Design / Project Architecture 12

4.1 High-Level Design 12

4.2 Detailed Design 13

4.3 Data Flow Diagrams 14

4.4 Database Design 15
5. Implementation 17
5.1 Module Description 17
5.2 Code Snippets 19
5.3 Integration 21
6. Testing 22
6.1 Test Plan 22
6.2 Test Cases 22
6.3 Results 23
6.4 Bug Fixes 23
7. Results and Discussion 24
7.1 Output Screenshots 24
7.2 Analysis of Results 24
7.3 Performance Evaluation 25
8. Conclusion and Future Scope 26
8.1 Summary 26
8.2 Limitations 26
8.3 Future Enhancements 27
References 28
Appendices 29

LIST OF FIGURES

Figure 4.1 System Architecture Diagram 12
Figure 4.2 Data Flow Diagram 14
Figure 4.3 Entity-Relationship Diagram.....15
Figure 5.1 Priority Algorithm Source Code.....19
Figure 5.2 Focus Score Calculation Source Code 20
Figure 5.3 Task API Endpoint Source Code.....20
Figure 7.1 Login Page.....24
Figure 7.2 Registration Page.....24
Figure 7.3 Dashboard.....24
Figure 7.4 Task Manager 24
Figure 7.5 Add New Task Form 25
Figure 7.6 Focus Timer Page 25
Figure 7.7 Analytics Page 25
Figure 7.8 Settings Page 25

LIST OF TABLES

Table 3.1 Tools and Technologies Used 10
Table 4.1 users Schema 15
Table 4.2 tasks Schema 15
Table 4.3 focus_sessions Schema 16
Table 4.4 daily_stats Schema 16
Table 5.1 Urgency Levels and Badge Colours 18
Table 6.1 Test Cases and Results 22
Table 7.1 Performance Evaluation 25

CHAPTER 1

INTRODUCTION

1.1 Background

Procrastination is something most students deal with at some point during their studies. It is not always about being lazy. A lot of the time the real problem is that there are too many tasks open at the same time and no clear idea of which one should be started first.

This problem is especially common in college life. In school, things are usually more structured. In college, a student has to manage assignments, practical work, class tests, project tasks, and preparation for exams all at the same time. When everything feels equally important, it becomes hard to choose the first step.

Productivity apps have existed for a long time. To-do lists, reminder apps, and calendar tools are very common. But many of them have one fundamental weakness, they show tasks, but they do not help in deciding which task deserves attention first. The user still has to make that decision manually, which is the exact problem causing the procrastination.

That is the gap Procrastify tries to solve. The idea behind this project was to create a system that automatically ranks tasks based on urgency and difficulty, helps the user focus on one task at a time, and gives useful analytics after each session.

1.2 Problem Statement

During our own experience as students, we noticed three specific and recurring problems.

The first problem is Prioritisation Paralysis. When a student has many assignments with different deadlines and different difficulty levels, it becomes genuinely difficult to decide where to start. Because of this confusion, students often delay starting any work at all.

The second problem is Lack of Focus. Even when a student decides what to work on, staying focused for a meaningful amount of time is difficult. Constant interruptions from social media, YouTube, chats, and general mental distraction break concentration before real progress is made.

The third problem is Lack of Productivity Awareness. Most students do not have real data about how they actually spend their study time. They may feel they studied a lot, but in reality a large portion of that time may have gone into distractions.

Procrastify was designed to address all three of these problems together in one integrated application.

1.3 Objectives

The main objectives of this project are:

- To build a task management system where tasks are automatically prioritised using a formula based on deadline and complexity.
- To provide a Pomodoro-based focus timer that records study sessions and promotes discipline.
- To create an analytics dashboard that shows daily and weekly productivity patterns through visual charts.
- To implement secure login and registration so each user has their own private data.
- To design the system architecture in a way that future features can be added without redesigning everything from scratch.

1.4 Scope of the Project

The present version of Procrastify includes user registration and login, task management, automatic task priority calculation, Pomodoro focus timer sessions, streak tracking, and a productivity analytics dashboard.

The current version does not include real-time app tracking, mobile application support, automatic distraction alerts, or full public cloud deployment. These features are planned and are described under future scope.

1.5 Methodology

The development process followed a straightforward, iterative approach. First, the main problems faced by students were identified and documented. After that, the required features were listed and the database schema was planned.

The backend was then developed using Flask and MySQL. After the backend became stable, the Streamlit frontend was created and connected to the API endpoints. Once both parts were working together, manual testing was done across all major modules.

This step-by-step approach kept the project manageable and helped avoid confusion during development.

1.6 Organisation of the Report

This report is divided into eight chapters. Chapter 1 introduces the project, its background, problem statement, and objectives. Chapter 2 covers the literature review and related work. Chapter 3 explains system analysis and project planning. Chapter 4 describes the system design and database structure. Chapter 5 covers implementation details and code. Chapter 6 explains the testing process and results. Chapter 7 presents output screenshots and analysis. Chapter 8 gives the conclusion and future scope of the project.

CHAPTER 2

LITERATURE REVIEW

2.1 Previous Work

Before building Procrastify, it was important to study what existing productivity tools and academic research already offered. The goal was not to repeat what was already done, but to find the gap and fill it.

Research on procrastination has consistently shown that it is not primarily a time management problem but a self-regulation problem. Steel (2007) described procrastination as a meta-analytic failure of self-regulation, noting that students delay tasks not out of laziness but due to poor task clarity and perceived overwhelming complexity.

Existing productivity tools fall broadly into two categories. The first category includes simple to-do apps like Todoist, Microsoft To-Do, and Google Tasks. These tools are effective at storing tasks but offer no intelligent ranking. The user must manually decide priority, which reintroduces the decision paralysis they were trying to escape.

The second category includes full-featured time management platforms like Notion, Trello, or Asana. These tools are powerful but come with a steep learning curve and are designed more for team collaboration than individual student use. A student who is already overwhelmed is unlikely to invest time in setting up a complex workflow system.

2.2 Related Studies

The Pomodoro Technique, developed by Francesco Cirillo, has been widely studied as a method for managing focus. The core principle is that working in short, defined intervals with regular breaks helps maintain mental energy and reduces the subjective difficulty of starting a task. Several academic papers have reported positive effects of structured work sessions on task completion rates.

Research in Human-Computer Interaction (HCI) has shown that users respond positively to gamification elements in productivity software. Streak tracking, progress indicators, and achievement levels tend to increase application engagement and help with long-term habit formation. This directly influenced the design of Procrastify's streak system.

In the area of task prioritisation, the Eisenhower Matrix and similar urgency-importance frameworks have been studied extensively. However, these frameworks require manual input and human judgment for every task. The gap in existing tools is the absence of automatic, formula-based prioritisation that adjusts continuously as deadlines approach.

2.3 Summary

The literature review made one clear point. Many existing tools solve only one part of the student productivity problem. Some track tasks, some track time, and some provide analytics. Very few combine all three in a single student-friendly application that requires minimal setup.

That gap led directly to the development of Procrastify. The goal was to create a unified system that handles prioritisation, focus timing, and analytics in one place, without requiring the user to make complex configuration decisions.

CHAPTER 3

SYSTEM ANALYSIS / PROJECT PLANNING

3.1 Requirements Gathering

Functional Requirements

The following functional requirements were identified based on the core problem statement:

- User registration with email, username, and securely hashed password.
- User login with JWT token generation for session management.
- Task creation with title, description, category, deadline, and complexity (1–5).
- Automatic priority score calculation on task creation and update.
- Task listing sorted by priority score in ascending order.
- Task editing, deletion, and completion marking.
- Pomodoro focus timer with 15, 25, 45, and 60-minute options.
- Session logging with start time, end time, and interruption status.
- Daily analytics showing Focus Score, productive time, and distraction time.
- Weekly trend chart showing focus patterns across the last 7 days.
- Streak tracking with level progression.
- Settings page for profile information and Pomodoro duration preference.

Non-Functional Requirements

The following non-functional requirements were set:

- Fast page loading on a standard college laptop.
- Secure password storage using bcrypt hashing.
- Token-based authentication using JWT for all protected routes.
- Clean, dark-themed user interface designed for minimal distraction.
- Application must run fully on a local system using XAMPP without cloud dependencies.

3.2 Feasibility Study

Technical Feasibility

The project was technically feasible because all required technologies are free, open-source, and available. Python, Flask, Streamlit, MySQL, and XAMPP can all run on a standard college laptop without special hardware.

Operational Feasibility

The project was operationally feasible because the final system is straightforward to use. A student only needs to register, add tasks, and start the timer. No advanced technical knowledge is required once the application is running.

Economic Feasibility

The project was economically feasible because the entire software stack is open-source. There were no licensing or hosting costs involved during development.

3.3 Tools and Technologies Used

Tool / Technology	Purpose
Python	Main programming language for backend and frontend logic
Flask	Backend API development and route management
Streamlit	Frontend web interface (5-page application)
MySQL	Relational database for persistent data storage
XAMPP	Local server environment for Apache and MySQL
JWT (PyJWT)	User authentication via token-based sessions
bcrypt	Secure password hashing and verification
VS Code	Code editor used throughout development
Postman	API endpoint testing and debugging
GitHub	Version control and code collaboration
Plotly	Interactive charts for analytics dashboard

Tool / Technology	Purpose
python-dotenv	Environment variable and configuration management

Table 3.1 — Tools and Technologies Used

3.4 Risk Analysis

Several risks were identified and managed during development:

- Database Connection Failure: XAMPP connection drops were mitigated by testing with retry logic and standardising startup procedures.
- JWT Handling Issues: Token expiry and cross-module authentication bugs were caught early through Postman testing of every protected route.
- Timer Inaccuracy: Browser tab-switching caused visual timer drift. This was resolved by using actual server-side timestamps instead of relying on the client-side countdown alone.
- Scope Creep: There was a real risk of attempting too many features. The project scope was consciously limited to five core modules to ensure a complete, working product.

CHAPTER 4

SYSTEM DESIGN / PROJECT ARCHITECTURE

4.1 High-Level Design

Procrastify follows a three-tier client-server architecture:

- Tier 1 = Frontend (Streamlit): Handles the user interface, form inputs, chart rendering, and display logic across six pages.
- Tier 2 = Backend (Flask REST API): Handles all business logic, authentication, the priority algorithm, and database operations through 18 defined API endpoints.
- Tier 3 = Database (MySQL): Stores all persistent data across 7 normalised relational tables.

The frontend never communicates with the database directly. Every request goes through the Flask backend, which processes the logic and returns a JSON response. This separation keeps the system more secure and maintainable.

[INSERT SCREENSHOT HERE]

Figure 4.1 — System Architecture Diagram (Frontend → Backend → Database)

approx. 6–8 cm tall

Figure 4.1 — System Architecture Diagram

4.2 Detailed Design

The frontend is divided into six separate Streamlit pages:

- Login / Registration Page: User authentication entry point.
- Dashboard: Displays a personalised greeting, today's Focus Score, top priority tasks, current streak, and a summary of today's sessions.

- Task Manager: Allows task creation, editing, deletion, and status filtering (Pending / Completed / All). Shows colour-coded priority badges.
- Focus Timer: A Pomodoro countdown with a Focus Lock mode that hides the navigation bar during active sessions.
- Analytics: A donut chart showing productive vs. distraction time, a weekly focus trend line chart, and a recent session history log.
- Settings: Profile information, preferred Pomodoro duration, and logout.

The backend Flask API is organised into 18 endpoints covering authentication, task management, focus session handling, analytics retrieval, and user settings.

4.3 Data Flow Diagrams

Primary Data Flow

The user interacts with the Streamlit frontend by registering, adding tasks, and starting focus sessions. Each action triggers an HTTP request to the Flask API. The API validates the JWT token in the request header, processes the business logic (such as calculating a priority score), and communicates with the MySQL database. The processed response is sent back to Streamlit as JSON, which renders it visually for the user.

Input Path: Student → Streamlit UI → Flask REST API → MySQL Database

Output Path: MySQL Database → Flask API → Streamlit UI → Student

[INSERT SCREENSHOT HERE]
Figure 4.2 — Data Flow Diagram
approx. 6–8 cm tall

Figure 4.2 — Data Flow Diagram

4.4 Database Design

The database consists of 7 normalised MySQL tables. The core tables are described below.

[INSERT SCREENSHOT HERE]
Figure 4.3 — Entity-Relationship Diagram
approx. 6–8 cm tall

Figure 4.3 — Entity-Relationship Diagram

users table

Column	Type	Description
user_id	INT (PK)	Unique user identifier
username	VARCHAR	User's display name
email	VARCHAR	User's email address
password_hash	VARCHAR	bcrypt-hashed password
created_at	DATETIME	Account registration timestamp
last_login	DATETIME	Last successful login time

Table 4.1 — users Schema

tasks table

Column	Type	Description
task_id	INT (PK)	Unique task identifier
user_id	INT (FK)	Reference to users table
title	VARCHAR	Task title
description	TEXT	Task description
category	VARCHAR	Study / Personal / Work / Other
deadline	DATE	Task due date

Column	Type	Description
complexity	INT	Difficulty rating from 1 to 5
priority_score	FLOAT	Calculated score (days / complexity)
status	VARCHAR	Pending or Completed

Table 4.2 — tasks Schema

focus_sessions table

Column	Type	Description
session_id	INT (PK)	Unique session identifier
user_id	INT (FK)	Reference to users table
task_id	INT (FK)	Linked task for this session
start_time	DATETIME	Session start timestamp
end_time	DATETIME	Session end timestamp
duration_minutes	INT	Timer duration selected
focus_score	FLOAT	Session-level focus score
was_interrupted	BOOLEAN	Whether session was interrupted

Table 4.3 — focus_sessions Schema

daily_stats table

Column	Type	Description
stat_id	INT (PK)	Unique record identifier
user_id	INT (FK)	Reference to users table
date	DATE	Date of the record
productive_time	INT	Total productive minutes logged
distraction_time	INT	Total distraction minutes logged

Column	Type	Description
sessions_completed	INT	Number of sessions completed
daily_focus_score	FLOAT	Average focus score for the day

Table 4.4 — daily_stats Schema

Additional tables: streaks (streak tracking and level progression), app_usage (future scope), and app_categories (future scope).

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

5.1.1 Authentication Module

The authentication module handles user registration and login. During registration, the entered password is hashed using bcrypt before being stored in the database. During login, the entered password is compared against the stored hash. If credentials match, a JWT token is generated and returned to the frontend. This token is stored in Streamlit's session state and attached to the header of every subsequent API request.

5.1.2 Task Management Module

This module allows users to create, edit, delete, and view tasks. Each task includes a title, description, category, deadline, and complexity rating. When a task is created or updated, the priority calculation runs immediately on the backend and the result is stored in the database.

Tasks are displayed to the user sorted by priority score in ascending order. Lower scores appear at the top because they represent higher urgency. Each task card shows a colour-coded badge indicating its urgency level.

5.1.3 Priority Algorithm Module

The priority algorithm is the core innovation of Procrastify. The formula is:

Priority Score = Days Until Deadline ÷ Complexity

A lower score indicates higher urgency. If the deadline has already passed, the score is automatically set to 0 and the task is marked OVERDUE regardless of complexity. The urgency levels and their corresponding score thresholds are:

Urgency Level	Score Condition	Badge Colour
OVERDUE	Days remaining ≤ 0 (Score = 0)	Red
HIGH	Score ≤ 3	Orange-Red

Urgency Level	Score Condition	Badge Colour
MEDIUM	Score ≤ 8	Orange
LOW	Score ≤ 15	Yellow
RELAXED	Score > 15	Green

Table 5.1 — Urgency Levels and Badge Colours

Example calculations to illustrate the logic:

- "Submit Assignment" = 3 days left, Complexity 1 → Score = 3 ÷ 1 = 3.0 → HIGH
- "BCA Project Notes" = 12 days left, Complexity 3 → Score = 12 ÷ 3 = 4.0 → MEDIUM
- "Personal Reading" = 30 days left, Complexity 2 → Score = 30 ÷ 2 = 15.0 → LOW
- "Missed Deadline Task" = 0 or negative days → Score = 0 → OVERDUE

The system also calculates the Focus Score for analytics:

Focus Score = (Productive Time ÷ Total Tracked Time) × 100

Focus Score thresholds: ≥ 80% = Excellent, ≥ 60% = Good, ≥ 40% = Fair, < 40% = Poor.

5.1.4 Focus Timer Module

The timer module provides a Pomodoro-style countdown integrated into the Streamlit interface using JavaScript. It supports sessions of 15 (default), 25, 45, and 60 minutes. A Focus Lock mode hides the sidebar navigation during an active session to reduce distraction.

After a session ends, the user self-reports whether the session was completed or interrupted. The session is then logged to the database with its start timestamp, end timestamp, duration, and focus score.

5.1.5 Analytics Module

The analytics module calculates and displays the following metrics:

- Daily Focus Score: The percentage of tracked time that was productive.
- Productive vs. Distraction Breakdown: A donut chart showing the split between focused work and distraction time.
- Weekly Focus Trend: A line chart plotting the daily focus score over the last 7 days.
- Session History: A log of recently completed focus sessions with their scores.

- Streak Level: Current streak count and level label (Starting, Consistent, On Fire, Legendary).

5.2 Code Snippets

The screenshots below show the actual source code from our project. Each snippet is captured directly from the working codebase.

5.2.1 Priority Algorithm — priority.py

[INSERT SCREENSHOT HERE]
Figure 5.1 — Source code snapshot of priority.py
approx. 6–8 cm tall

Figure 5.1 — Priority Algorithm (priority.py)

5.2.2 Focus Score Calculation

[INSERT SCREENSHOT HERE]
Figure 5.2 — Source code snapshot of the focus score calculation
approx. 6–8 cm tall

Figure 5.2 — Focus Score Calculation

5.2.3 Task API Endpoint — routes.py

[INSERT SCREENSHOT HERE]

Figure 5.3 — Source code snapshot of the /api/tasks endpoint

approx. 6–8 cm tall

Figure 5.3 — Task API Endpoint (routes.py)

5.3 Integration

The frontend and backend run as two separate processes that communicate through HTTP. The Flask backend runs on localhost:5000 and the Streamlit frontend runs on localhost:8501. XAMPP maintains the MySQL database in the background.

The standard startup sequence for the application is:

1. Start Apache and MySQL services in the XAMPP control panel.
2. Run the Flask backend: python backend/app.py
3. Run the Streamlit frontend: streamlit run streamlit/app.py
4. Open the browser and navigate to localhost:8501.

Every request from Streamlit to Flask includes the JWT token in the Authorization header. The Flask middleware validates the token before processing any protected route.

CHAPTER 6

TESTING

6.1 Test Plan

Testing was carried out in two stages. In the first stage, individual modules were tested separately to verify that each component worked as expected in isolation. In the second stage, the complete application was tested end-to-end, from the login screen through to the analytics dashboard.

Most testing was manual. API endpoints were tested using Postman to verify request and response formats. User flows were tested directly in the browser, working through real usage scenarios.

6.2 Test Cases

Test Case	Input	Expected Output	Result
TC-01: Registration	Valid username, email, password	Account created, redirect to login	Pass
TC-02: Login (valid)	Correct credentials	JWT token issued, dashboard opens	Pass
TC-03: Login (invalid)	Wrong password	Error message displayed	Pass
TC-04: Create task (future date)	Deadline in future, complexity 3	Priority score calculated correctly	Pass
TC-05: Overdue task	Deadline in past	Score = 0, marked OVERDUE in red	Pass
TC-06: Edit task	Update deadline and complexity	Priority score recalculated	Pass
TC-07: Delete task	Select existing task	Task removed from list	Pass

Test Case	Input	Expected Output	Result
TC-08: Start timer	Select 15-minute session	Countdown starts correctly	Pass
TC-09: Complete session	Timer reaches zero	Session logged to database	Pass
TC-10: Analytics view	Session data exists in DB	Focus Score and chart render correctly	Pass

Table 6.1 — Test Cases and Results

6.3 Results

All core modules worked successfully during testing. Registration, login, task management, timer handling, streak tracking, and analytics all behaved as expected under normal test conditions.

The priority calculation produced logically correct results in all tested cases, including edge cases with very high or very low complexity values and tasks due on the same day.

6.4 Bug Fixes

Several issues were identified and resolved during the testing phase:

- Timer tab-switching drift: The timer count became inaccurate when the user switched to another browser tab. This was fixed by switching from a purely visual countdown to a timestamp-based approach that compares the current time against the stored session start time.
- Task list not refreshing after edit: After editing a task, the page was showing stale values. This was fixed by forcing a Streamlit page rerun after any task update.
- bcrypt package conflict: A dependency version conflict appeared during setup on one machine. This was resolved by standardising the package versions in a requirements.txt file.
- JWT token expiry on long sessions: Extended use caused token expiry mid-session. This was addressed by implementing a 7-day token validity period aligned with standard session practices.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Output Screenshots

The following figures show the final working application. Screenshots illustrate all major pages of the Procrastify system.

[INSERT SCREENSHOT HERE]
Figure 7.1 — Login Page
approx. 6–8 cm tall

Figure 7.1 — Login Page

[INSERT SCREENSHOT HERE]
Figure 7.2 — Registration Page
approx. 6–8 cm tall

Figure 7.2 — Registration Page

[INSERT SCREENSHOT HERE]
Figure 7.3 — Dashboard: showing Focus Score, streak, and top priority tasks
approx. 6–8 cm tall

Figure 7.3 — Dashboard

[INSERT SCREENSHOT HERE]

Figure 7.4 — Task Manager: showing colour-coded priority badges and task cards

approx. 6–8 cm tall

Figure 7.4 — Task Manager

[INSERT SCREENSHOT HERE]

Figure 7.5 — Add New Task Form

approx. 6–8 cm tall

Figure 7.5 — Add New Task Form

[INSERT SCREENSHOT HERE]

Figure 7.6 — Focus Timer Page: active Pomodoro session with Focus Lock

approx. 6–8 cm tall

Figure 7.6 — Focus Timer Page

[INSERT SCREENSHOT HERE]

Figure 7.7 — Analytics Page: donut chart and weekly focus trend

approx. 6–8 cm tall

Figure 7.7 — Analytics Page

[INSERT SCREENSHOT HERE]

Figure 7.8 — Settings Page: profile and Pomodoro preference

approx. 6–8 cm tall

Figure 7.8 — Settings Page

7.2 Analysis of Results

The final application successfully achieved all the goals defined at the beginning of the project. It combines task management, automatic prioritisation, a focus timer, streak tracking, and a productivity analytics dashboard in one cohesive system.

The priority formula worked effectively because it balanced two variables, deadline pressure and task difficulty, rather than relying on deadline alone. This made the task list actively more useful than a standard to-do list. In testing, the formula consistently placed the most genuinely urgent tasks at the top of the list.

The focus timer added structure to study sessions, and the Focus Lock feature reduced distraction during sessions. Users who tested the application noted that having a clear countdown made it easier to commit to working without checking their phone.

The analytics dashboard made the app feel meaningful because it gave feedback instead of just recording data silently. Seeing a concrete Focus Score and weekly trend gave users a way to measure their actual study behaviour rather than relying on subjective feelings about productivity.

7.3 Performance Evaluation

Metric	Observed Result
Page loading speed	Fast on local system (< 2 seconds)
Task creation	Successful, priority score appears immediately
Priority calculation	Instant, no perceptible delay

Metric	Observed Result
Timer response	Smooth, accurate to timestamp in normal use
Analytics rendering	Charts rendered correctly with data present
JWT authentication	All protected routes secured correctly
Session logging	All completed sessions recorded accurately

Table 7.1 — Performance Evaluation

For a locally deployed student project, the overall performance was highly satisfactory. All core functions operated without errors in standard usage conditions.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Summary

Procrastify began as a project idea rooted in a real problem faced by students every semester. Instead of building another simple to-do app, the focus was on creating a system that could actively reduce the confusion of what to work on first.

The final system includes secure authentication, a full task management interface, a two-variable priority algorithm, a Pomodoro-based focus timer with Focus Lock, streak tracking with gamified levels, and a real-time productivity analytics dashboard.

The development process also provided substantial practical learning in database design, REST API architecture, JWT authentication, bcrypt hashing, frontend-backend integration, and the process of debugging a full-stack web application.

The project demonstrates that even a student-built application can incorporate meaningful intelligence, and in this case, automatic task prioritisation makes everyday use more productive and less stressful.

8.2 Limitations

The application currently runs on a local system using XAMPP and is not deployed to a public cloud server. This means it can only be used on the development machine unless the environment is replicated.

The Focus Score calculation still depends partly on user-reported information. The user manually logs distraction time rather than the system detecting it automatically. This is a known limitation and is planned for a future version.

The streak system and analytics are accurate but rely on consistent daily usage. Users who skip days without logging any sessions will see their streak reset, which could be discouraging in some cases.

8.3 Future Enhancements

Several enhancements are already planned and documented for future versions of Procrastify:

- Mobile Application: A React Native or Flutter app for iOS and Android that keeps the same priority logic and analytics but on a mobile-native interface.
- Cloud Deployment: Deploying the application to a cloud server (such as Render or Railway) so it is accessible from any device with an internet connection.
- Real-Time App Tracking: Automatic detection of active window time, allowing the system to classify distraction time without manual user input.
- Automated Distraction Alerts: System notifications when a user spends more than a threshold of time on classified distraction apps such as Instagram or YouTube.
- AI/ML Integration: A machine learning model that predicts task complexity based on category, past completion patterns, and historical session data.
- Team Mode: Multi-user project boards with task delegation, shared task lists, and team productivity analytics.
- Calendar Synchronisation: Integration with Google Calendar and other scheduling tools to pull deadlines automatically.

REFERENCES

[1] Steel, P. (2007). The nature of procrastination: A meta-analytic and theoretical review of quintessential self-regulatory failure. Psychological Bulletin, 133(1), 65–94.

[2] Cirillo, F. (2006). The Pomodoro Technique. FC Garage.

[3] Pallets Projects. (2023). Flask Documentation. https://flask.palletsprojects.com/

[4] Streamlit Inc. (2023). Streamlit Documentation. https://docs.streamlit.io/

[5] Auth0. (2023). Introduction to JSON Web Tokens. https://jwt.io/introduction

[6] Bcrypt. (2023). bcrypt – Password Hashing Library for Python. https://pypi.org/project/bcrypt/

[7] Oracle Corporation. (2023). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/

[8] Apache Friends. (2023). XAMPP Documentation. https://www.apachefriends.org/

[9] Plotly Technologies Inc. (2023). Plotly Python Open Source Graphing Library. https://plotly.com/python/

[10] Foord, M. (2023). PyJWT Documentation. https://pyjwt.readthedocs.io/

APPENDICES

Appendix A: Source Code Files

The following files constitute the complete source code of the Procrastify project:

Backend (Flask API)

- backend/app.py — Application entry point and Flask configuration
- backend/routes.py — All 18 REST API endpoint definitions
- backend/priority.py — Priority score and focus score calculation logic
- backend/auth.py — JWT and bcrypt authentication helpers
- backend/database.py — MySQL connection and query utilities

Frontend (Streamlit)

- streamlit/app.py — Main application entry point and login/registration UI
- streamlit/pages/1_Dashboard.py — Dashboard page with summary stats
- streamlit/pages/2_Tasks.py — Task manager with priority display
- streamlit/pages/3_Timer.py — Pomodoro timer with Focus Lock
- streamlit/pages/4_Analytics.py — Charts and focus score display
- streamlit/pages/5_Settings.py — User profile and preferences

Appendix B: User Manual

To install and run Procrastify on a local machine:

5. Install XAMPP and ensure Apache and MySQL services are available.
6. Install Python 3.9 or higher.
7. Install required Python packages: pip install -r requirements.txt
8. Start Apache and MySQL in the XAMPP Control Panel.
9. Import the database schema in phpMyAdmin by running the provided SQL script.
10. Run the Flask backend: python backend/app.py
11. Run the Streamlit frontend: streamlit run streamlit/app.py

12. Open a browser and navigate to http://localhost:8501

Appendix C: Glossary

Term	Definition
JWT	JSON Web Token — a compact, URL-safe means of representing claims transferred between frontend and backend for authentication.
bcrypt	A password-hashing function that applies a salt to the password before hashing, making brute-force attacks significantly harder.
REST API	Representational State Transfer API — a communication method between frontend and backend using HTTP requests and JSON responses.
Pomodoro	A time management technique involving 25-minute work intervals separated by short breaks. Procrastify supports 15, 25, 45, and 60-minute variants.
Priority Score	The output of the formula: Days Until Deadline ÷ Complexity. A lower score indicates higher urgency.
Focus Score	A percentage calculated as: (Productive Time ÷ Total Tracked Time) × 100. Measures the quality of a study session.
XAMPP	A free, open-source cross-platform web server solution providing Apache and MySQL for local development.
Flask	A lightweight Python web framework used to build the Procrastify REST API backend.
Streamlit	An open-source Python library for building interactive web applications, used as the Procrastify frontend.
Streak	A consecutive-day counter that tracks daily application usage. Increases user motivation through gamification.